# Comparative Analysis of Classification Algorithms for Bank Marketing Prediction

**Case Study 6**

Roll No: 160124733365

Name: B. Madhav

Section: CSE-3

Year: 3rd Year

Subject: Fundamentals of Data Science (FDS)

# Methodology

1. **Data understanding:** loaded bank-full.csv (45,211 rows, 17 columns, ';' delimiter), checked data types, statistics, 'unknown' values, duplicates and target distribution.
2. **EDA:** target bar chart, subscription rate by job, month and previous outcome, and box plots of age, balance and campaign against the target.
3. **Preprocessing:** dropped *duration* (only known after the call), one-hot encoded categorical columns, standardised features, stratified 80:20 train-test split.
4. **Modelling:** Logistic Regression, Decision Tree, KNN (k=15) and Random Forest trained on the same split, with balanced class weights where supported.
5. **Evaluation:** accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices and ROC curves; the best model was chosen using F1, recall and ROC-AUC, not accuracy alone.
6. **Interpretation:** Random Forest feature importances and business recommendations.

*Tools: Python, pandas, NumPy, matplotlib, seaborn, scikit-learn. AI assistance (Claude) was used while writing the code.*

# Case Study 6: Comparative Analysis of Classification Algorithms for Bank Marketing Prediction

**Roll No:** 160124733365 | **Subject:** Fundamentals of Data Science (FDS)

**Problem statement:** A Portuguese bank runs telephone campaigns to sell term deposits. Using customer and campaign data, build and compare Logistic Regression, Decision Tree, KNN and Random Forest models to predict whether a customer will subscribe (`y = yes`), and recommend the best model for targeting customers.

*Note: AI assistance (Claude) was used to help write this code; the outputs were run and checked by me.*

## Task 1: Data Understanding and Preprocessing

The file `bank-full.csv` (UCI Bank Marketing) uses `;` as delimiter.

```
Shape: (45211, 17)
```

| | age | job | marital | education | default | balance | housing | loan | contact | day | month |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 58 | management | married | tertiary | no | 2143 | yes | no | unknown | 5 | may |
| **1** | 44 | technician | single | secondary | no | 29 | yes | no | unknown | 5 | may |
| **2** | 33 | entrepreneur | married | secondary | no | 2 | yes | yes | unknown | 5 | may |
| **3** | 47 | blue-collar | married | unknown | no | 1506 | yes | no | unknown | 5 | may |
| **4** | 33 | unknown | single | unknown | no | 1 | no | no | unknown | 5 | may |
| **5** | 35 | management | married | tertiary | no | 231 | yes | no | unknown | 5 | may |
| **6** | 28 | management | single | tertiary | no | 447 | yes | yes | unknown | 5 | may |
| **7** | 42 | entrepreneur | divorced | tertiary | yes | 2 | yes | no | unknown | 5 | may |
| **8** | 58 | retired | married | primary | no | 121 | yes | no | unknown | 5 | may |
| **9** | 43 | technician | single | secondary | no | 593 | yes | no | unknown | 5 | may |

The dataset has 45,211 customer records and 17 columns (16 predictors + target `y`).

```
age          int64
job            str
marital        str
education      str
default        str
balance      int64
housing        str
loan           str
contact        str
day          int64
month          str
duration     int64
campaign     int64
pdays        int64
previous     int64
poutcome       str
y              str
dtype: object
```

| | age | balance | day | duration | campaign | pdays | |
|---|---|---|---|---|---|---|---|
| **count** | 45211.000000 | 45211.000000 | 45211.000000 | 45211.000000 | 45211.000000 | 45211.000000 | 452 |
| **mean** | 40.936210 | 1362.272058 | 15.806419 | 258.163080 | 2.763841 | 40.197828 | |
| **std** | 10.618762 | 3044.765829 | 8.322476 | 257.527812 | 3.098021 | 100.128746 | |
| **min** | 18.000000 | -8019.000000 | 1.000000 | 0.000000 | 1.000000 | -1.000000 | |
| **25%** | 33.000000 | 72.000000 | 8.000000 | 103.000000 | 1.000000 | -1.000000 | |
| **50%** | 39.000000 | 448.000000 | 16.000000 | 180.000000 | 2.000000 | -1.000000 | |
| **75%** | 48.000000 | 1428.000000 | 21.000000 | 319.000000 | 3.000000 | -1.000000 | |
| **max** | 95.000000 | 102127.000000 | 31.000000 | 4918.000000 | 63.000000 | 871.000000 | 2 |

Customers are 18 to 95 years old (mean about 41). `balance` is highly skewed with negative values and a max over 100,000. `campaign` (number of calls in this campaign) is usually 1 to 3. `pdays = -1` means the customer was never contacted before, which is true for most customers.

```
Numerical  : ['age', 'balance', 'day', 'duration', 'campaign', 'pdays', 'previous']
Categorical: ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contac
t', 'month', 'poutcome']
Target     : y

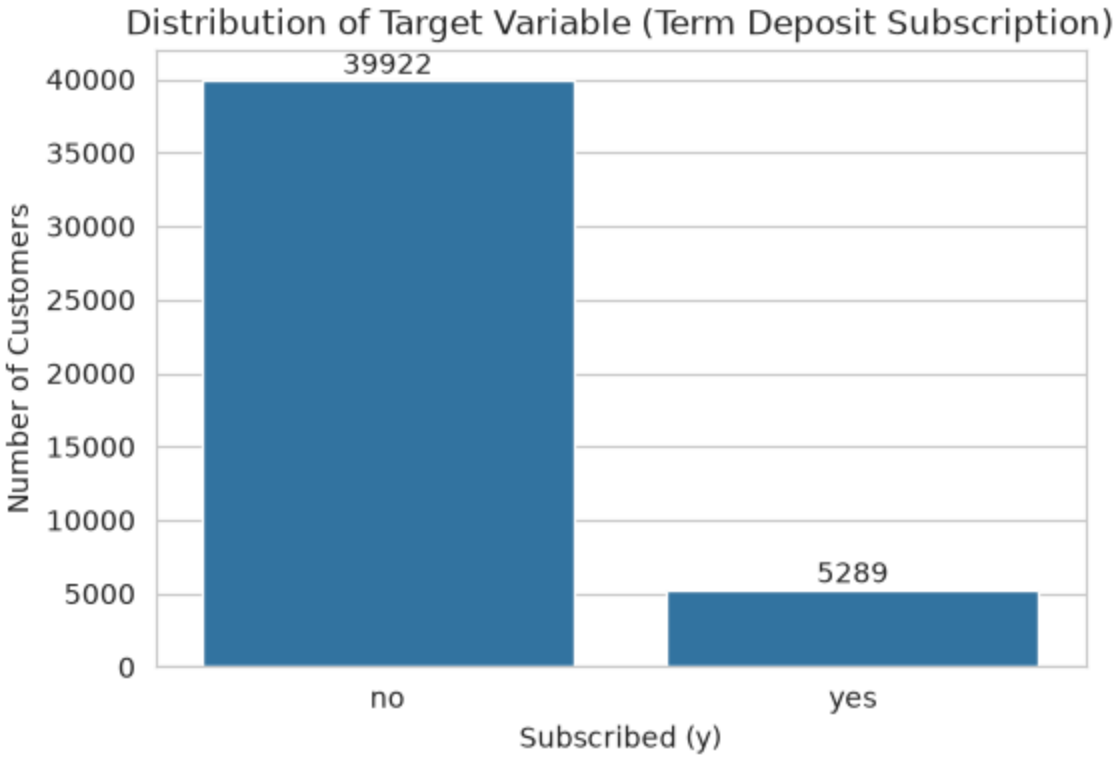
Missing values: 0
Duplicate rows: 0

'unknown' entries per column:
job            288
marital          0
education     1857
default          0
housing          0
loan             0
contact      13020
month            0
poutcome     36959
dtype: int64
```

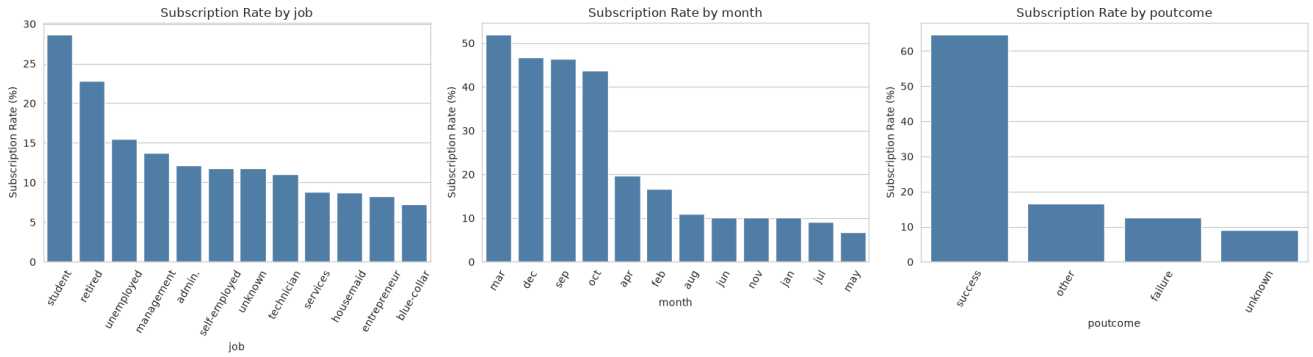
There are no null values and no duplicate rows, but missing information is stored as the text `'unknown'`. `poutcome` is unknown for most rows (no previous campaign) and `contact` for about 29%. Since `unknown` itself carries meaning (e.g. not contacted before), it is kept as a separate category instead of being dropped.

# Task 2: Exploratory Data Analysis

```
     Count  Percentage
y
no   39922        88.3
yes   5289        11.7
```




**Interpretation:** 39,922 customers (88.3%) did not subscribe and only 5,289 (11.7%) subscribed. The data is highly imbalanced, so a model that always says "no" would already get 88% accuracy. This is why accuracy alone cannot be used to pick the best model.

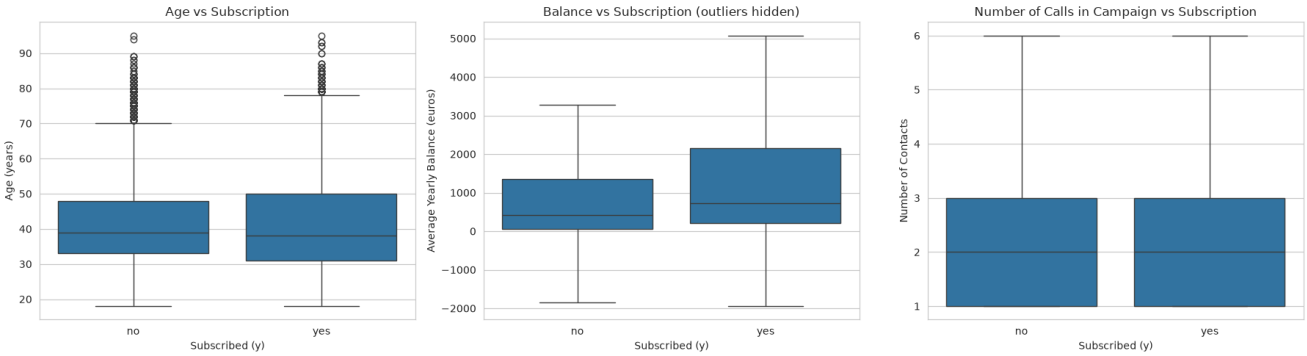



```
{'student': 28.7, 'retired': 22.8, 'unemployed': 15.5, 'management': 13.8, 'admi
n.': 12.2, 'unknown': 11.8, 'self-employed': 11.8, 'technician': 11.1, 'services':
8.9, 'housemaid': 8.8, 'entrepreneur': 8.3, 'blue-collar': 7.3}
{'mar': 52.0, 'dec': 46.7, 'sep': 46.5, 'oct': 43.8, 'apr': 19.7, 'feb': 16.6, 'au
g': 11.0, 'jun': 10.2, 'nov': 10.2, 'jan': 10.1, 'jul': 9.1, 'may': 6.7}
{'success': 64.7, 'other': 16.7, 'failure': 12.6, 'unknown': 9.2}
```

**Interpretation:**

- **Job:** students (28.7%) and retired people (22.8%) subscribe far more often than blue-collar workers (7.3%) and entrepreneurs (8.3%).
- **Month:** March (52%), December (46.7%), September (46.5%) and October (43.8%) have very high success rates, even though few calls were made then. May has the most calls but the lowest rate (6.7%).

- **Previous outcome:** customers whose previous campaign was a success subscribe 64.7% of the time, compared with only 9.2% for customers never contacted before.




| y | age | balance | campaign | previous |
|---|---|---|---|---|
| no | 39.0 | 417.0 | 2.0 | 0.0 |
| yes | 38.0 | 733.0 | 2.0 | 0.0 |

**Interpretation:** Median age is almost the same for both groups (38 vs 39), but the age range of subscribers is wider, covering more young and older (retired) customers. Subscribers have a higher median balance (733 vs 417 euros), so customers with more money are more likely to put it in a term deposit. Subscribers were usually contacted fewer times in this campaign; calling the same customer many times does not increase success.

## Three key observations from EDA

1. **A previous success is the strongest sign:** 64.7% of customers with `poutcome = success` subscribed again, about 7 times the overall rate of 11.7%.
2. **Timing matters:** March, September, October and December have success rates above 40%, while the busy month of May has only 6.7%. The bank calls the most when results are poorest.
3. **Customer profile matters:** students and retired customers, and customers with higher balances, are much more likely to subscribe than blue-collar workers.

## Encoding, scaling and train-test split

`duration` is dropped because it is only known after the call ends, so it cannot be used for realistic prediction before contacting a customer.

```
Features after encoding: 41
Train: (36168, 41)  Test: (9043, 41)
Subscribed share in train / test: 0.117 0.117
```

Categorical columns are one-hot encoded (giving 41 features) and all features are standardised, which matters for Logistic Regression and especially KNN (distance based). The split is 80:20 and stratified so both sets keep the same ~11.7% subscription rate. The same split is used for all four models.

# Task 3: Classification Model Development

Logistic Regression, Decision Tree and Random Forest use `class_weight='balanced'` so the small 'yes' class is not ignored. The tree depth is limited to reduce overfitting. KNN uses k = 15.

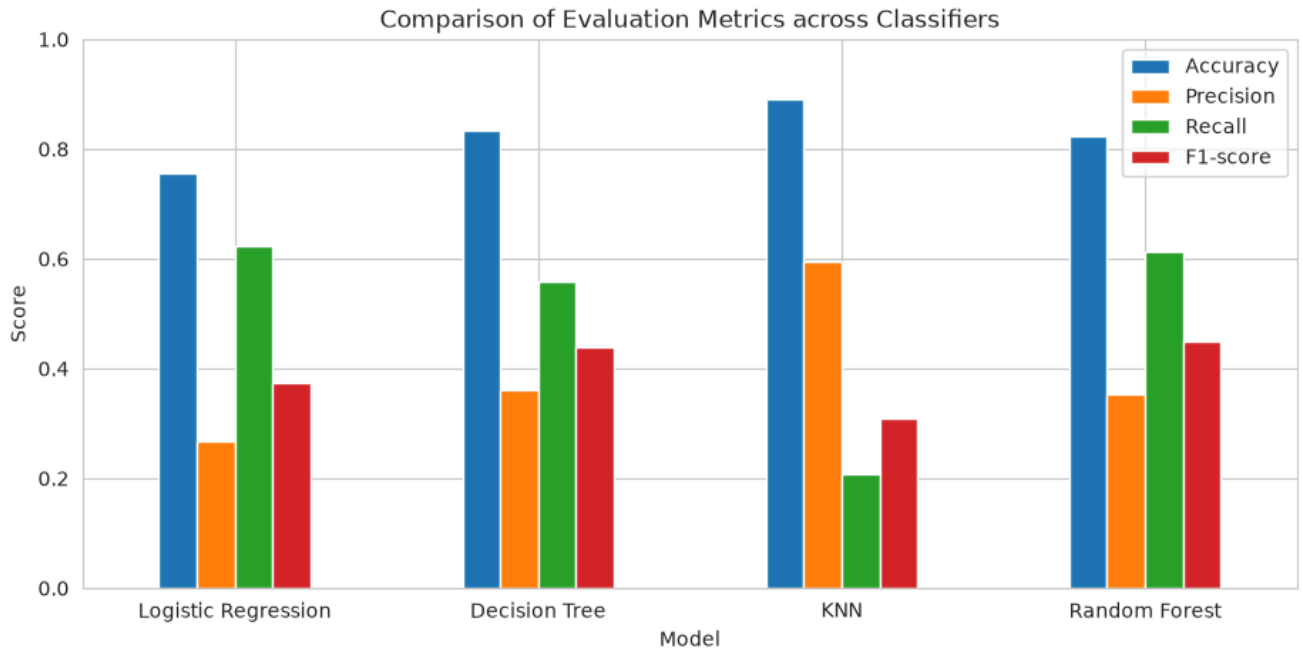
| Model | Accuracy | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| **Logistic Regression** | 0.7557 | 0.2669 | 0.6229 | 0.3737 | 0.7722 |
| **Decision Tree** | 0.8326 | 0.3608 | 0.5586 | 0.4384 | 0.7629 |
| **KNN** | 0.8909 | 0.5962 | 0.2079 | 0.3083 | 0.7441 |
| **Random Forest** | 0.8237 | 0.3540 | 0.6144 | 0.4492 | 0.7976 |

# Task 4: Model Performance Comparison and Visualization

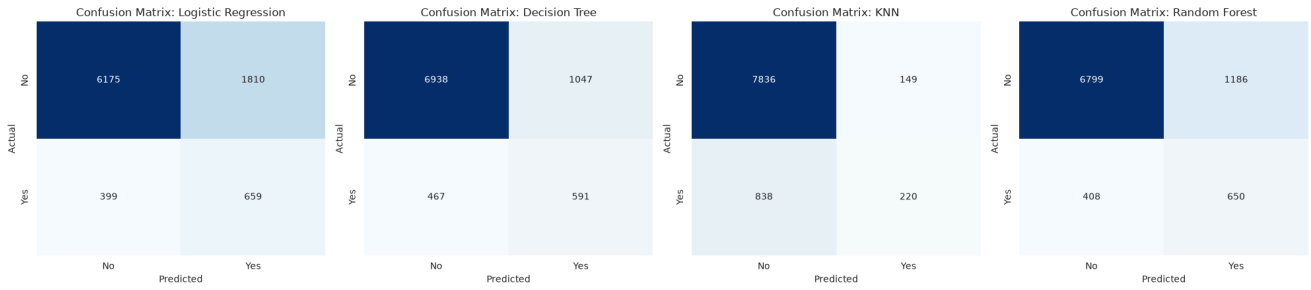
## Final performance comparison table

| Model | Accuracy | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| **Random Forest** | 0.8237 | 0.3540 | 0.6144 | 0.4492 | 0.7976 |
| **Decision Tree** | 0.8326 | 0.3608 | 0.5586 | 0.4384 | 0.7629 |
| **Logistic Regression** | 0.7557 | 0.2669 | 0.6229 | 0.3737 | 0.7722 |
| **KNN** | 0.8909 | 0.5962 | 0.2079 | 0.3083 | 0.7441 |

**Interpretation:** KNN has the highest accuracy (89.1%) and precision (0.60), but its recall is only 0.21: it finds just 1 in 5 real subscribers. This happens because it mostly predicts "no", the majority class. Random Forest has the best F1-score (0.449) and the best ROC-AUC (0.798), with high recall (0.614). Logistic Regression has the highest recall (0.623) but the lowest precision (0.267), so it produces many false alarms. Decision Tree is in between.




**Interpretation:** The chart shows the trade-off between precision and recall. KNN's tall accuracy and precision bars hide a very short recall bar. Random Forest and Decision Tree have the most balanced bars, and Random Forest is slightly ahead on recall and F1-score. All models have fairly low precision because subscribers are rare and are hard to separate without call duration.



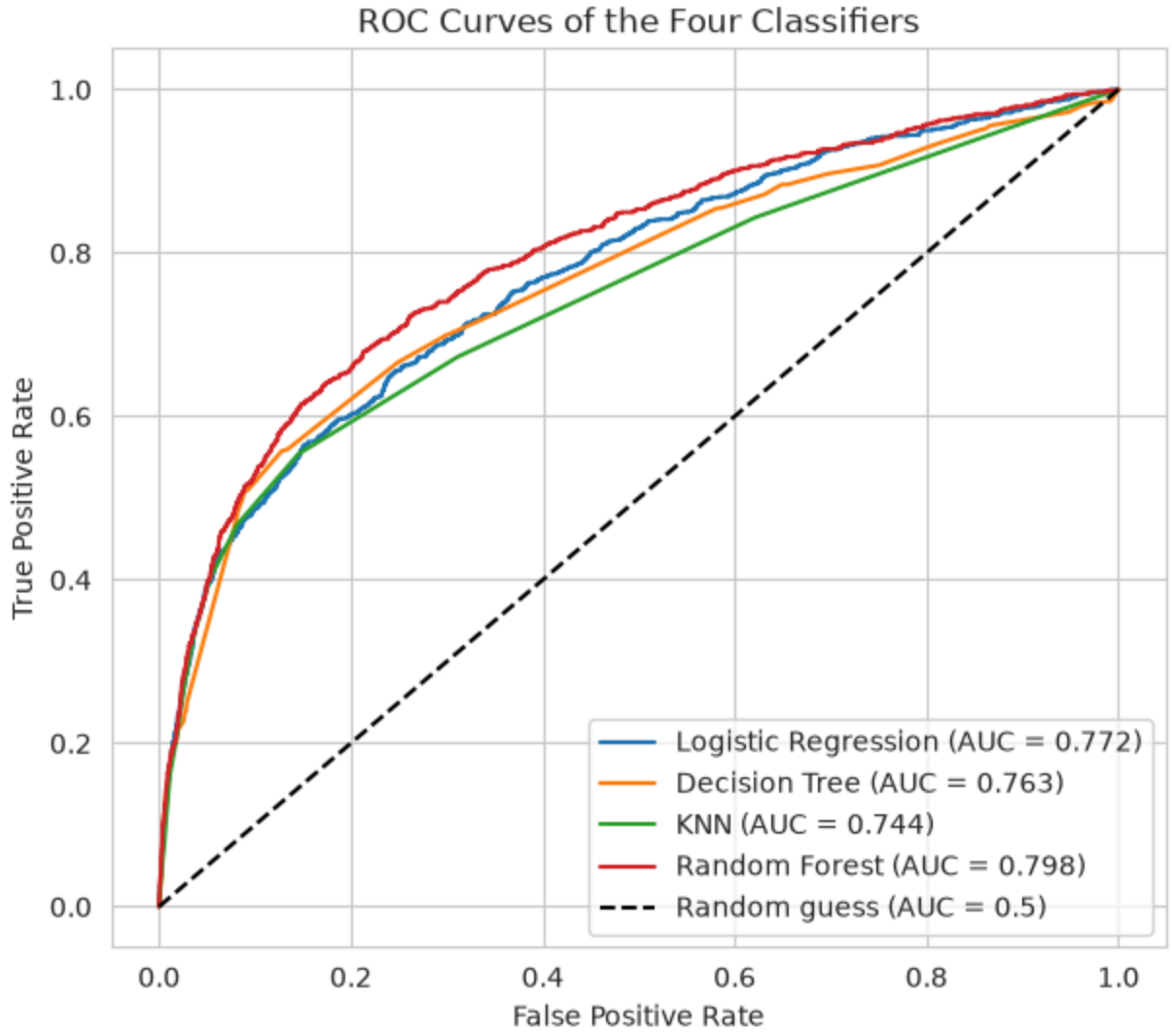

```
Logistic Regression   TN= 6175 FP= 1810 FN= 399 TP= 659
Decision Tree         TN= 6938 FP= 1047 FN= 467 TP= 591
KNN                   TN= 7836 FP=  149 FN= 838 TP= 220
Random Forest         TN= 6799 FP= 1186 FN= 408 TP= 650
```

**Interpretation of errors (9,043 test customers, 1,058 actual subscribers):**

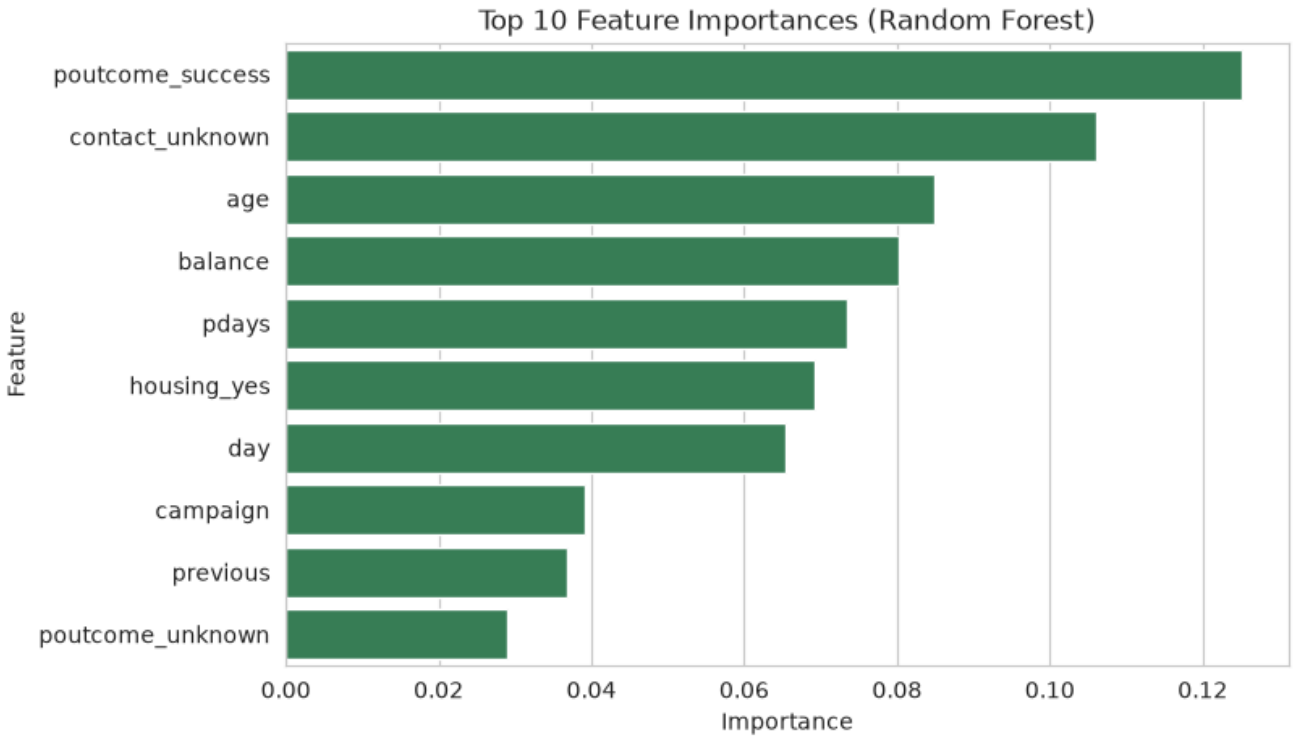
- **Logistic Regression:** catches 659 subscribers but makes 1,810 false positives (wasted calls).
- **Decision Tree:** 591 subscribers caught, 1,047 false positives, 467 missed.
- **KNN:** only 149 false positives, but it misses 838 of 1,058 subscribers (false negatives), losing most of the potential customers.
- **Random Forest:** catches 650 subscribers (almost as many as Logistic Regression) with 624 fewer false positives (1,186).

For the bank, a **false negative** (missing a customer who would have subscribed) loses revenue, while a **false positive** only costs one phone call. So missing subscribers is more expensive, and recall is

very important.




**Interpretation:** All four curves are above the random line, so every model has some predictive power even without call duration. Random Forest has the highest curve and AUC (0.798), followed by Logistic Regression (0.772), Decision Tree (0.763) and KNN (0.744). This means Random Forest is best at ranking likely subscribers above non-subscribers over all thresholds.




**Interpretation:** The most important features for Random Forest are previous campaign success ( `poutcome_success` ), unknown contact type ( `contact_unknown` , which has low success), age, balance, days since last contact ( `pdays` ) and housing loan. This matches the EDA: past behaviour and financial situation are the best signs of a future subscription.

# Task 5: Interpretation and Business Recommendations

## Best model: Random Forest

- It has the **highest F1-score (0.449)** and **highest ROC-AUC (0.798)** of the four models.
- Its **recall (0.614)** is almost equal to the best (Logistic Regression, 0.623), but it has **624 fewer false positives**, so precision is better (0.354 vs 0.267).
- KNN looks best on accuracy (89.1%), but that is misleading on imbalanced data: it misses 79% of subscribers.
- Random Forest also handles non-linear relationships and mixed features well and gives feature importances that the bank can understand.

## Business recommendations

1. **Rank customers by predicted probability and call the top ones first.** Instead of calling everyone, the bank can focus on customers with high scores (e.g. past successes, students, retired, higher balance). This finds about 61% of subscribers while skipping most uninterested customers, saving time and cost.

2. **Change campaign timing and contact strategy.** Run more calls in March, September, October and December, and fewer in May. Limit repeated calls to the same customer, because success does not increase after the first few contacts. Re-contact customers who said yes in earlier campaigns.

## Limitations

1. **Class imbalance and low precision:** only 11.7% are subscribers, and even the best model's precision is 0.35, so about 2 of 3 predicted "yes" customers will still say no. The probability threshold could be tuned to suit the bank's call budget.
2. **Limited features and no tuning:** `duration` was correctly removed, and the remaining data has no information like income or online behaviour. Hyperparameters (k, tree depth, number of trees) were chosen by hand, not by cross-validation, so the results may be slightly better or worse with tuning. The data is also from an older (2008 to 2010) Portuguese campaign and may not generalise to other banks or time periods.

## Conclusion

Four classifiers were compared on the same 80:20 split without the `duration` feature. **Random Forest gave the best balance between finding subscribers and avoiding wasted calls** (F1 = 0.449, ROC-AUC = 0.798) and is recommended for targeting the bank's term deposit campaigns.